

Synthio Tutorial

Feature	Synthio Object	What It Controls
Waveform	Wave	Timbre (shape of sound)
Note	Note	Frequency + waveform + modulation
Envelope	Envelope	Loudness over time (ADSR)
Filter	Filter	Harmonics (brightness)
LFO	LFO	Modulation (pitch/volume/filter)
Synth Engine	Synthesizer	Plays notes + manages DSP
Mixer	Mixer	Volume + routing

General script setup

- Import libraries & Set variables
- Setup output
- Setup mixer
- Setup envelope
- Setup synth
- Setup filters
- Setup note
- Execute play
- Execute loop using a while true loop

Synthio Tutorial

Import Libraries & set variables

```
import board
import audiobusio
import audiomixer
import synthio
import array
import math
import time

SAMPLE_RATE = 22050
CHANNEL_COUNT = 2
BUFFER_SIZE = 2048
```

Set output (board dependant)

```
audio = audiobusio.I2SOut(
    bit_clock=board.GP27,      # BCLK
    word_select=board.GP28,    # LRC
    data=board.GP26,          # DIN
)
```

Synthio Tutorial

Mixer

Controls overall volume and mixes voices.

Define the mixer

```
mixer = audiomixer.Mixer(  
    voice_count=1,  
    sample_rate=22050,  
    channel_count=1,  
    bits_per_sample=16,  
    samples_signed=True,  
)
```

Play Synthio through the mixer

```
mixer.voice[0].play(synth)
```

Synthio Tutorial

Synthesizer

The main engine that plays notes and processes audio. Key properties:

- `sample_rate` — how many samples per second (affects quality + CPU load)
- Holds notes, waves, filters, envelopes, LFOs

Define the synth

```
synth = synthio.Synthesizer(sample_rate=22050, channel_count=2)
```

Synthio Tutorial

Wavetables

A wavetable is a list/array of 16-bit samples describing one cycle of a waveform.

Create a wavetable

```
import array
```

```
wavetable = array.array("h", [0] * 256)
```

Fill it with values

```
wavetable[i] = int(math.sin(angle) * 32767)
```

Turn it into a Synthio Wave

```
wave = synthio.Wave(wavetable)
```

Synthio Tutorial

Musical Notes

A note is a playable sound with a frequency and waveform.

Define a note

```
note = synthio.Note(frequency=220, waveform=wave)
```

Play / Stop

```
synth.press(note)
```

```
synth.release(note)
```

Synthio Tutorial

Envelopes (ADSR)

Shape loudness over time. What an ADSR does:

- Attack — how quickly a sound starts
- Decay — how it settles after attack
- Sustain — level while held
- Release — fade-out after release

Define the envelope

```
env = synthio.Envelope(attack_time=0.1, decay_time=0.2, sustain_level=0.7, release_time=0.3)
```

Attach to a note:

```
note = synthio.Note(frequency=220, waveform=wave, envelope=env)
```

Synthio Tutorial

Filters

Shape timbre by removing or emphasizing harmonics.

Define a Low-pass filter

```
lpf = synthio.Filter(low_pass=1200, resonance=1.5)
```

```
hpf = synthio.Filter(high_pass=600, resonance=1.5)
```

```
lpf = synthio.Filter(band_pass=1000, resonance=1.5)
```

Attach to a note:

```
note = synthio.Note(frequency=220, waveform=wave, filter=lpf)
```

Change cutoff dynamically

```
note.filter.low_pass = 3000
```

Synthio Tutorial

LFO (Low Frequency Oscillator)

Modulates pitch, amplitude, or filter cutoff. Common uses:

- Vibrato (pitch)
- Tremolo (volume)
- Filter sweeps

Create an LFO

```
lfo = synthio.LFO(  
    wave=synthio.Wave(wavetable),  
    frequency=5.0, # Hz  
    amplitude=5.0 # modulation depth  
)
```

Attach to a note:

```
note = synthio.Note(  
    frequency=220,  
    waveform=wave,  
    lfo=lfo  
)
```

Synthio Tutorial

APPENDIX

Common wavetables

sine wave, just like in trig class

```
wave_sine = np.array(np.sin(np.linspace(0, 2*np.pi, NUM, endpoint=False)) * VOL, dtype=np.int16)
```

sawtooth wave, looks like a downward ramp

```
wave_saw = np.linspace(VOL, -VOL, num=NUM, dtype=np.int16)
```

square wave, like the default

```
wave_square = np.concatenate((np.ones(NUM // 2, dtype=np.int16) * VOL, np.zeros(NUM // 2, dtype=np.int16) * -VOL))
```

'noise' wave made with random numbers (not a very good random noise)

```
wave_noise = np.array([random.randint(-VOL, VOL) for i in range(NUM)], dtype=np.int16)
```

Synthio Tutorial

Using a WAV File as a Wavetable

To load a WAV file as a wavetable, it must be:

1. 16-bit sample width
2. Mono (1 channel)
3. Raw waveform data must be divisible into 256-sample chunks

Load file

```
FNAME = " /wavetables/BRAIDS01.WAV"
```

```
WAVE_LEN = 256    # standard wavetable size
```

```
my_wave = adafruit_wave.open(FNAME)
```

Validate format (required)

```
if my_wave.getsampwidth() != 2 or my_wave.getnchannels() != 1:
```

```
    raise ValueError("WAV must be 16-bit mono")
```

Total frames in file

```
total_frames = my_wave.getnframes()
```

Number of single-cycle waves inside the file

```
num_waves = total_frames // WAVE_LEN
```

Synthio Tutorial

Buffer for one wave

```
wave_buf = np.zeros(WAVE_LEN, dtype=np.int16)

def load_wave(position):
    # Load one 256-sample chunk from the WAV file.
    my_wave.setpos(position * WAVE_LEN)
    raw = my_wave.readframes(WAVE_LEN)
    wave_buf[:] = np.frombuffer(raw, dtype=np.int16)
    return synthio.Wave(wave_buf)
```

Start with wave 0

```
wave = load_wave(0)
note = synthio.Note(frequency=110, waveform=wave)
mixer.voice[0].play(synth)
synth.press(note)
```

OPTIONAL: Scan through a wavetable

```
pos = 0
while True:
    pos = (pos + 1) % num_waves
    note.waveform = load_wave(pos)
    time.sleep(0.05)
```